atividade_curta_1

March 13, 2026

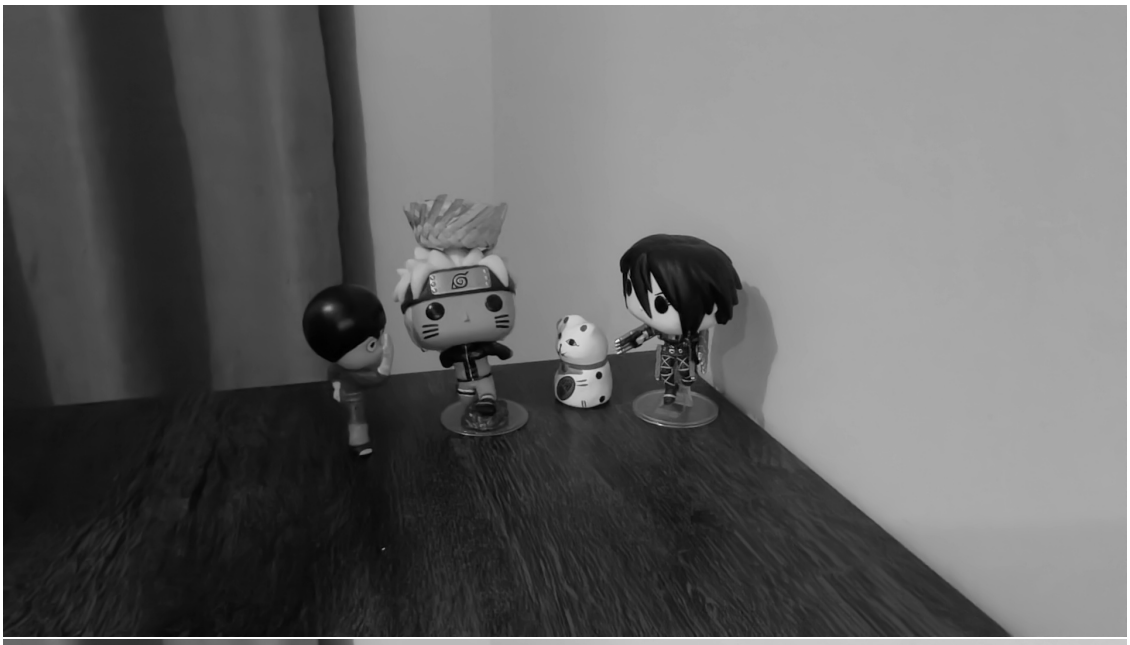
[7]:
```
import cv2 as cv
import PIL
import numpy as np

print('NumPy Version: ', np.__version__)
print('OpenCV Version: ', cv.__version__)
print('PIL Version: ', PIL.__version__)
```

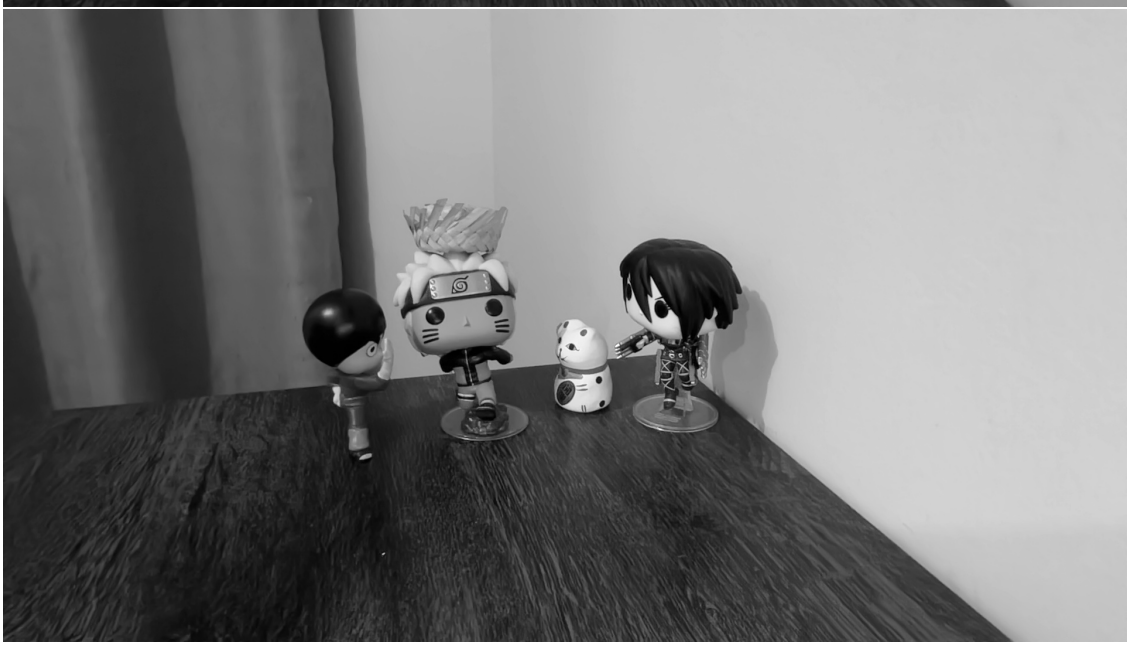
```
NumPy Version:  2.4.3
OpenCV Version:  4.13.0
PIL Version:  12.1.1
```

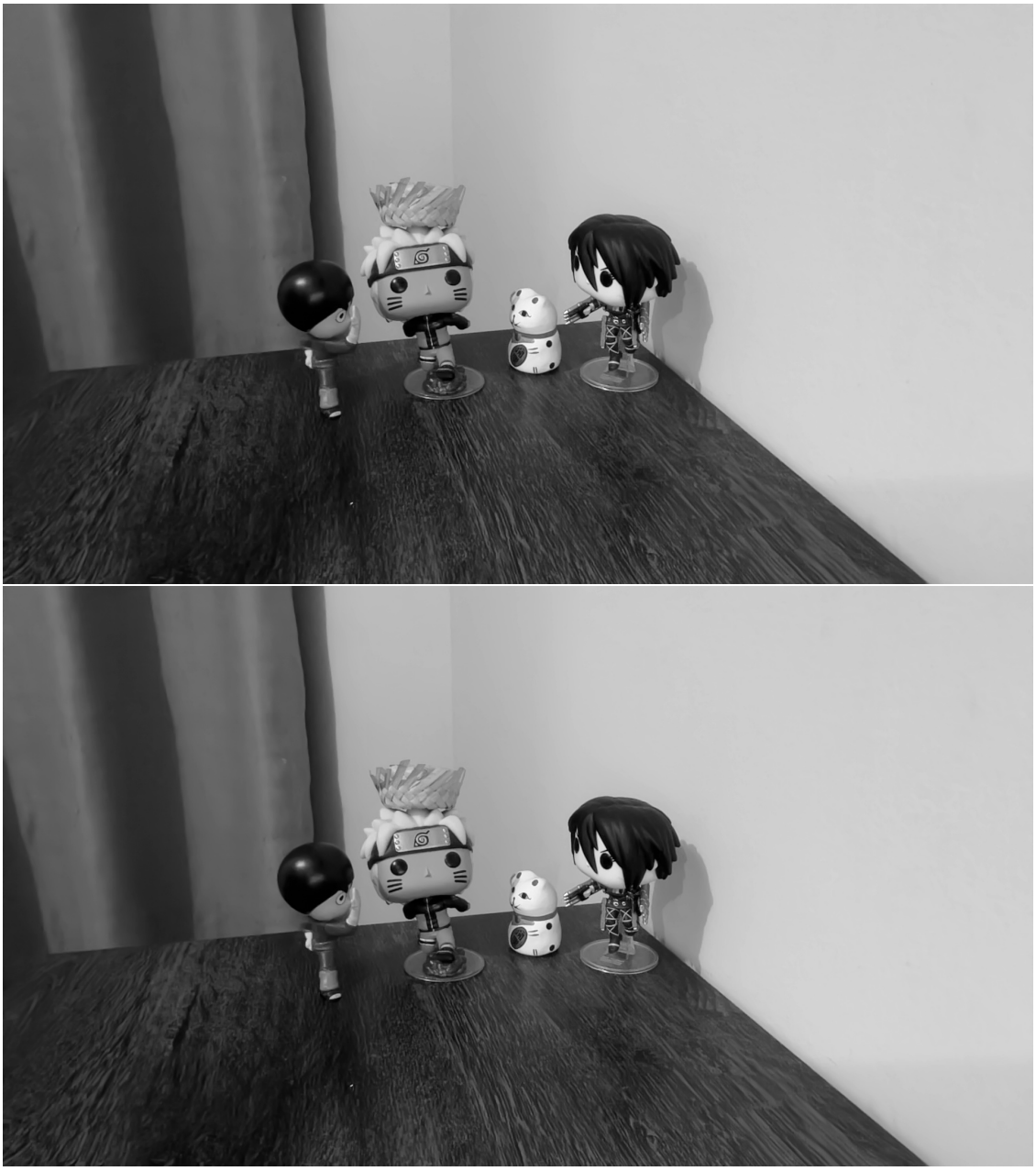
[52]:
```
# captura das imagens
import time

def getImages():
     # seleciona a câmera do notebook
    cap = cv.VideoCapture(1)

    # verifica se a conexão com a câmera foi estabelecida
    if not cap.isOpened():
        print("Não foi possível acessar a câmera")
        exit()

    # descarta os primeiros frames para evitar ruídos devido a inicialização da␣
 ↪câmera
    for _ in range(10):
        cap.read()

    pictures = []
    for i in range(10):
        # função para captura
        ret, frame = cap.read()

        # verifica se o frame foi capturado com sucesso
        if ret:
            filename = f"foto_webcam_{i}.jpg"
```

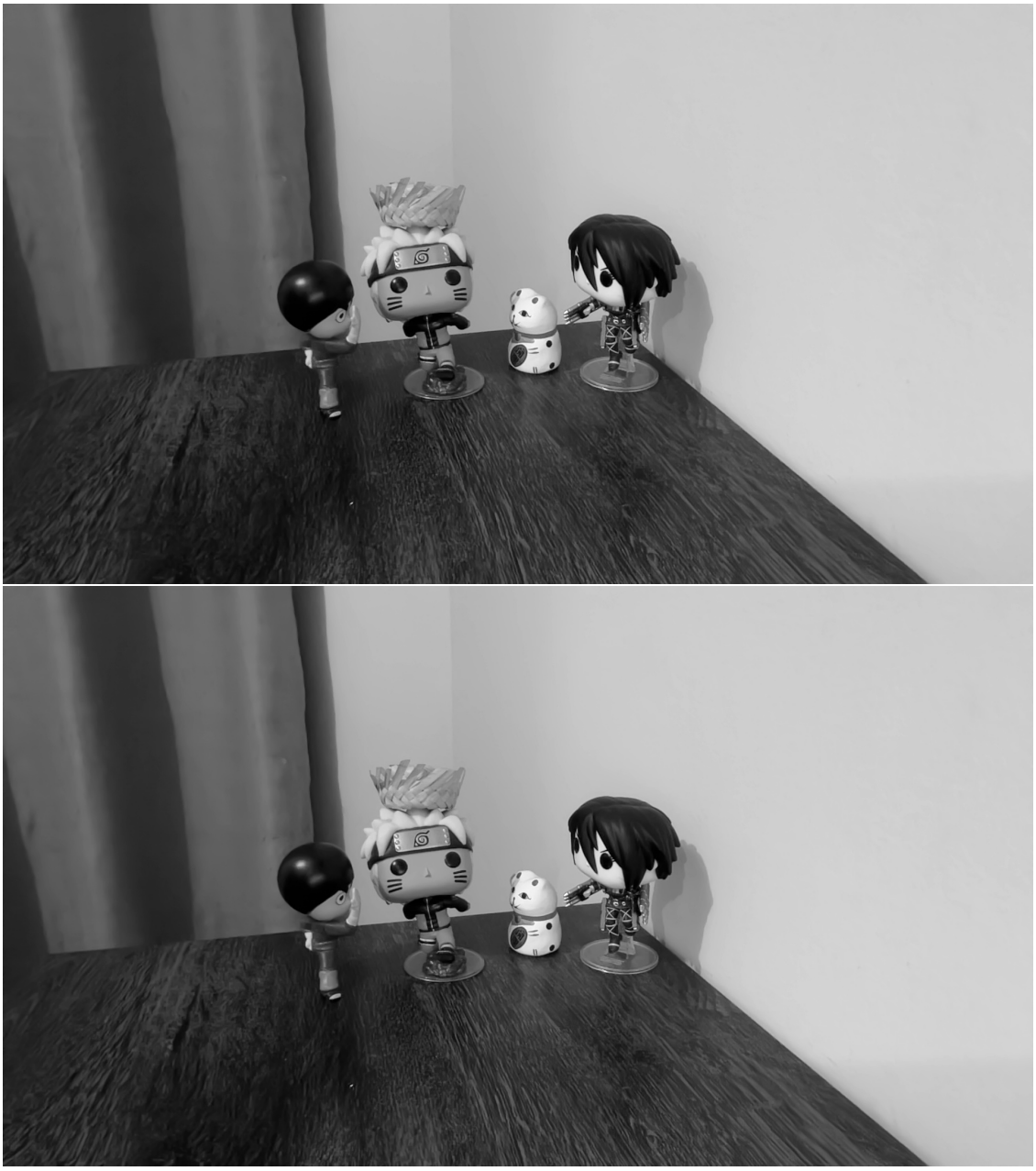
```
            gray = cv.cvtColor(frame, cv.COLOR_BGR2GRAY) # converte a foto␣
 ↪captura para tons de cinza
            cv.imwrite(filename, gray)
            pictures.append(gray)
            time.sleep(1)

    #encerra a conexão com a câmera
    cap.release()
    return pictures

pictures = getImages()
```

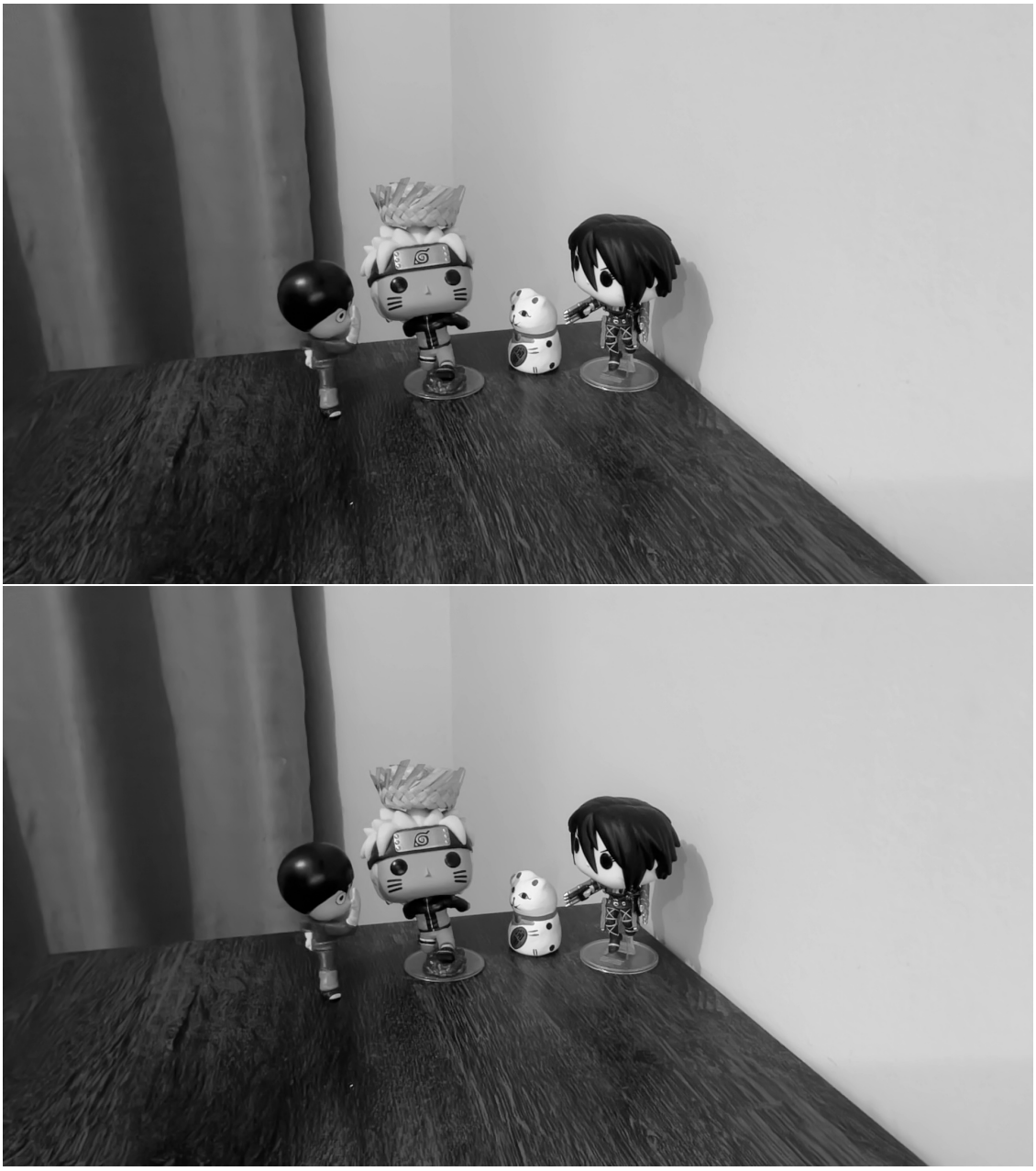
0.1 Imagens Originais

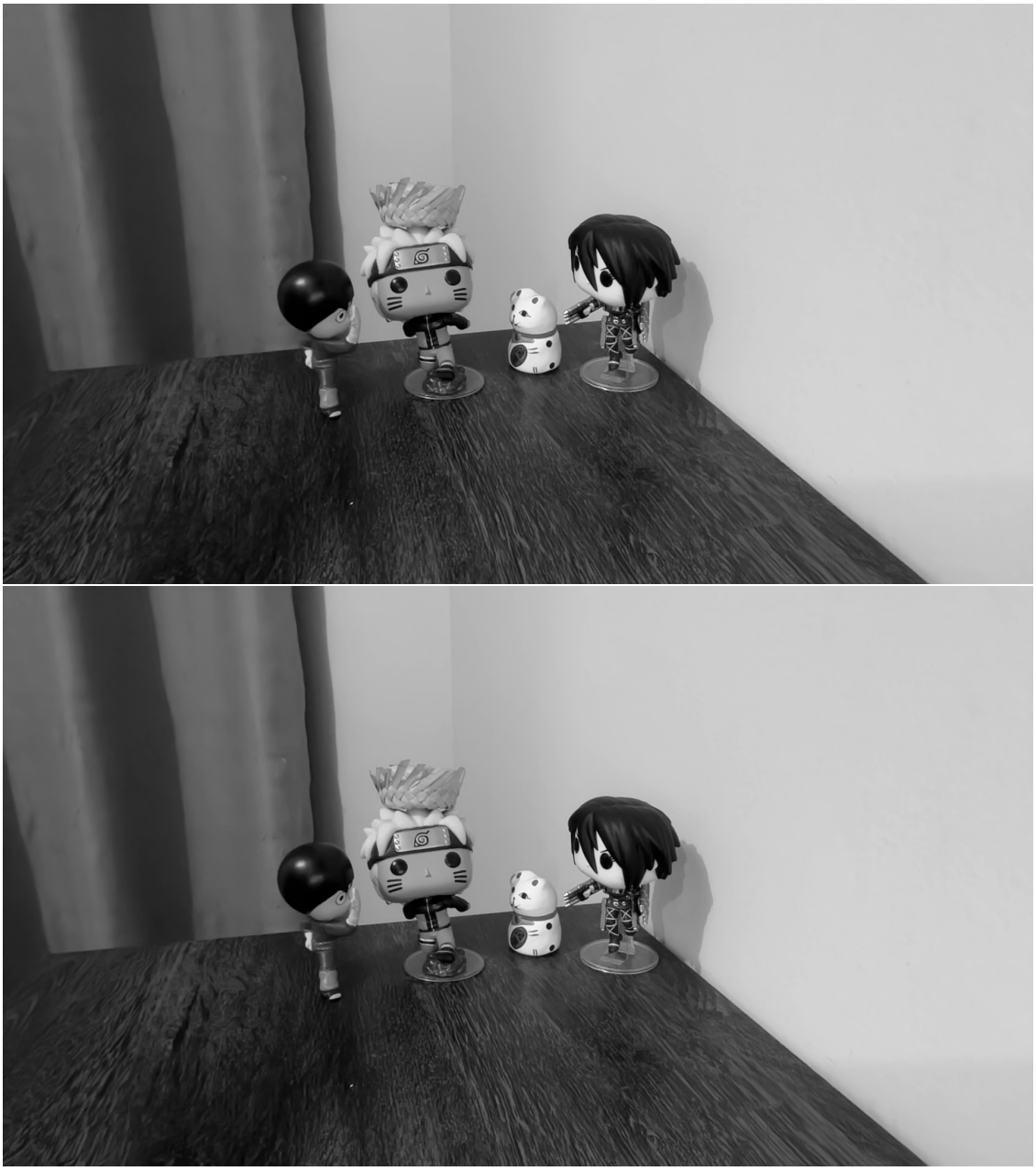












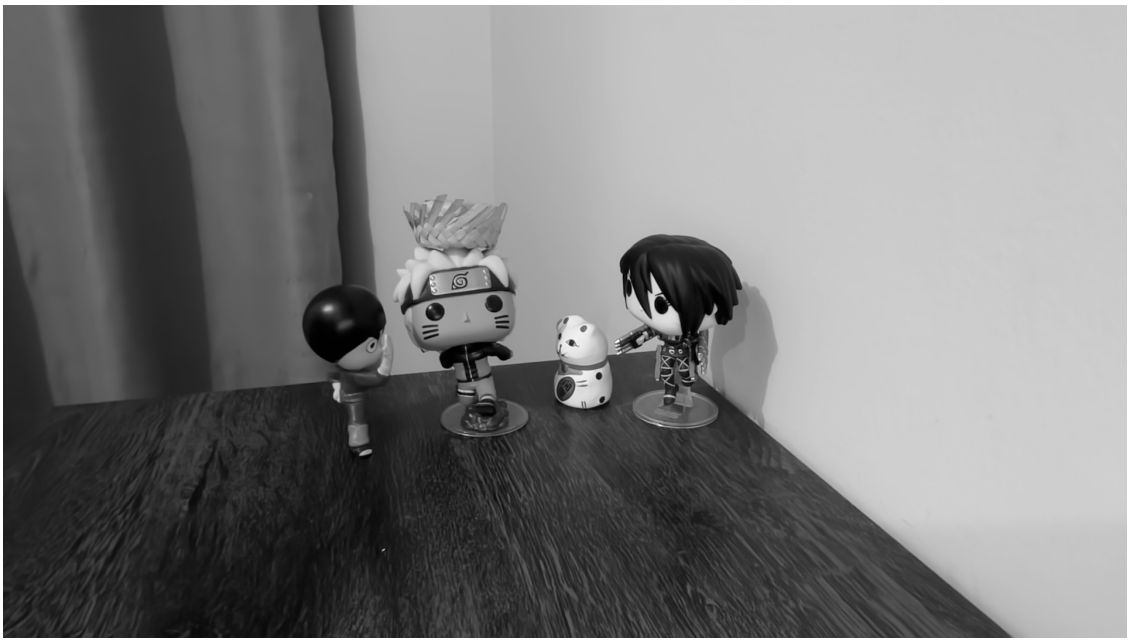
[53]:
```python
# calculo da diferença

def accumulated_difference(pictures):
    # cria uma matriz de int zerada
    accumulatedImg = np.zeros_like(pictures[0], dtype=np.int32)

    # faz o calculo da diferença, elemento por elemento
    for p in range(len(pictures) - 1):
        diff_img = cv.absdiff(pictures[p], pictures[p+1])
        accumulatedImg += diff_img
```

```
    # salva e retorna a imagem
    cv.imwrite("diferenca_acumulada.jpg", accumulatedImg)
    return accumulatedImg

accumulatedImg = accumulated_difference(pictures)
```

0.2 Imagem de Diferença Acumulada

[54]:
```
def average_image(pictures):
    # cria uma matriz de int zerada
    sumImg = np.zeros_like(pictures[0], dtype=np.int32)

    # faz a média simples das imagens
    for p in range(len(pictures)):
        sumImg += pictures[p]

    avarageImg = sumImg / len(pictures)

    cv.imwrite("media.jpg", avarageImg)
    return avarageImg

avarageImg = average_image(pictures)
```

0.3 Imagem Média



[55]:
```python
def standard_deviation(avarageImg):
    # cria uma matriz de float zerada
    deviationImg = np.zeros_like(pictures[0], dtype=np.float64)

    # calcula o desvio padrão
    for p in range(len(pictures)):
        deviationImg += (pictures[p] - avarageImg)**2
    deviationImg = np.sqrt(deviationImg / len(pictures))

    cv.imwrite("desvio_padrao.jpg", deviationImg)
    return deviationImg

deviationImg = standard_deviation(avarageImg)
```

0.4 Imagem de Desvio Padrão